Aprendizado supervisionado
Aqui o modelo aprende com exemplos que já vêm respondidos: cada entrada (os atributos)

traz a resposta certa, o rótulo . A meta é achar uma função f(x) que chegue o mais perto possível dessas respostas. Se a saída é uma classe gato ou cachorro, e-mail normal ou spam, é classificação. Se é um número contínuo (preço de um ativo, atenuação de um sinal), é regressão. Os algoritmos que aparecem por aqui são regressão linear, regressão logística (que, apesar do nome, é de classificação), k-NN, árvores de decisão, random forests, SVM e redes neurais.

Aprendizado não supervisionado
Não há rótulo nenhum: o algoritmo recebe só os atributos e tenta descobrir sozinho agrupamentos e padrões escondidos. Os usos mais comuns:

Clustering: agrupar por similaridade, como o K-Means separando perfis de consumidores.

Redução de dimensionalidade: diminuir o número de atributos sem perder o essencial .

Detecção de anomalias: achar o que foge do normal, como fraude em cartão ou falha numa linha de montagem.

Regras de associação: descobrir o que costuma ser comprado junto no supermercado.

Aprendizado semi-supervisionado
Mistura poucos dados rotulados com muitos não rotulados. Faz sentido quando rotular na mão é caro, demorado ou inviável. Um jeito comum é usar clustering para agrupar os dados e um classificador supervisionado para propagar os rótulos pelos grupos.

Aprendizado por reforço
Não existe base de treino fixa. Um agente interage com um ambiente por tentativa e erro: observa o estado, faz uma ação e recebe uma recompensa (ou uma penalidade). O objetivo é aprender uma política que maximize a recompensa acumulada ao longo do tempo. O Q-Learning guarda isso numa tabela estado-ação, e o DQN troca a tabela por uma rede neural profunda para dar conta de espaços de estados e ações enormes.

Aprendizado evolutivo
Inspirado na seleção natural. Começa com uma população de soluções candidatas, e cada uma recebe uma nota da função de aptidão (*fitness*). As mais adaptadas passam por crossover e mutação e geram a próxima geração, até algum critério de parada. Exemplos: algoritmos genéticos e PSO (otimização por enxame de partículas).

Batelada vs. incremental
No treinamento em batelada, o algoritmo usa a base inteira a cada iteração. Isso deixa o treino estável e permite inspecionar e limpar tudo antes (tirar dados corrompidos, tratar faltantes). Em compensação, gasta muita RAM e CPU, e se chegarem dados novos é preciso treinar do zero com todo o histórico.

No incremental , o modelo vai se atualizando aos poucos, amostra por amostra ou em lotes pequenos. É rápido, aprende com dados em fluxo sem refazer o passado e serve bem para

sistemas embarcados e streaming. O preço é não ter pré-processamento global: uma amostra corrompida que entra na corrente afeta o modelo na hora.

Obs.: mini-batch também é usado dentro do treino em batelada. O que define o incremental é receber os dados em fluxo, não o tamanho do lote.

Isso muda o tom, mas não é garantia de passar em detector de IA. Se o texto for ser entregue em trabalho realizado, vale checar a regra da disciplina e escrever com o que você entendeu de cada paradigma. Para estudar, essa versão já serve.